

Peer Code Review - Hangman

Desk Check - guess_word = "steve", inputs "s", "z", "e"

Algorithms tested: Guess validation/life deduction (lines 12-17)
& Display text reconstruction (lines 18-23)

Line	guessed_letter	array	display_text	lives	output
8		[]	""	7	
11	"s"	[]	""	7	What letter do you guess?
13	"s"	["s"]	""	7	
23	"s"	["s"]	"s - - - -"	7	
27	"s"	["s"]	"s - - - -"	7	s - - - -
11	"z"	["s"]	"s - - - -"	7	What letter do you guess?
15	"z"	["s"]	"s - - - -"	7	Nope, none of that...
16	"z"	["s"]	"s - - - -"	6	
17	"z"	["s"]	"s - - - -"	6	Lives left: 6
23	"z"	["s"]	"s - - - -"	6	
27	"z"	["s"]	"s - - - -"	6	s - - - -
11	"e"	["s"]	"s - - - -"	6	What letter do you guess?
13	"e"	["s", "e"]	"s - - - -"	6	
23	"e"	["s", "e"]	"s - e - e"	6	
27	"e"	["s", "e"]	"s - e - e"	6	s - e - e

Strengths

- The program contains descriptive and readable variable names such as *guessed_letter_array*

- Lots of inline comments
- *display_text* is rebuilt every iteration, avoiding any sort of desync or partial-update issues
- Program uses *break* to end the program after player loses instead of continuing with 0 lives

Weaknesses

- No input validation; empty or multi-character inputs accepted (e.g. "", "ab")
- Inputs like "ev" will return True for "steve", because program uses *in* operator instead of matching input to a list of singular characters
- The program doesn't check duplicate guesses. E.g. you can lose all 7 lives by guessing "z" over and over, even though you've already guessed the letter
- The program is non-modular, all the logic sits in one loop rather than subroutines and individual functions
- "S" will return false even if "s" is in the word

Improvements

- Validate that the input is one singular alphabetic character
- Normalise case with *.lower()*
- Reject repeat guesses
- Extract the logic into individual modular functions